wsdm-cup-multilingual-chatbot-arena

3位 Solution

Rank	3
Team	PLaMo 1000000B
Author	RihanPiggy
Topic ID	567584
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/567584
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

入賞された皆さん、おめでとうございます。このような興味深いコンペを開催してくださった主催者の皆様に感謝します。LMSYSは私にとって初めてのLLMコンペで、銀メダルでした。今回のWSDM Cupでは金メダルを取ることができ、とても嬉しく思います。ここで私のsolutionを共有します。

学習コード: https://github.com/LIHANG-HONG/WSDM-Cup-3rd-place-solution 推論コード: https://www.kaggle.com/code/honglihang/wsdm-submission-nocot-public?scriptVersionId=228251584

TL;DR

- Eedi Rerank Modelの学習パイプラインを再実装しました。vllmで高速に推論するため、AutoModelForSequenceClassificationではなくAutoModelForCasualLMを使います。
- Qwen/Qwen2.5-14B-Instruct（post-pretrainあり）+ Phi4（post-pretrainなし）。異なるseedで学習したmodelをmergeしました。
- LMSYSの1位と同様に、72B modelでdistillationしました。ただし、72B modelは必須ではないことが分かりました。14B modelのlogitを使って「self-distillation」を行っても、同じCV scoreに到達できます。本当に重要なのは、soft logitがもたらすlabel cleaningの効果なのかもしれません。
- auto-roundで量子化しました。
- vllmで推論します。ensembleでは、まずtoken長でソートし、Qwen2.5-14Bで25%のTTAを行います。残りの推論時間はPhi4に使います。

前処理

fasttextを使ってpromptの言語を推定します。

学習パート

SFTTrainerのcompute_loss関数をoverrideし、Token "A" と "B" だけについてlossを計算します。

```
():
  ():
        labels = inputs[]
        labels[labels > ] = -
        inputs[] = labels
        _, outputs = ().compute_loss(model, inputs, return_outputs=,
num_items_in_batch=num_items_in_batch)
        logits = outputs.logits
        shift_logits = logits[..., :-, :].contiguous()
        shift_labels = labels[..., :].contiguous()

        logits_target = []
        labels_target = []
```

```
         i  ((shift_labels)):
            lbl = shift_labels[i].cpu().numpy()
            target_idx = np.where(lbl != -)[][]

            logits_target.append(shift_logits[i][target_idx][:])
            labels_target.append(shift_labels[i][target_idx]-)


        logits_target = torch.stack(logits_target, dim=)

        labels_target = torch.tensor(labels_target).to(outputs.logits.device)
        loss = F.cross_entropy(logits_target, labels_target)
         return_choice_logit:
             (loss, outputs, logits_target)  return_outputs  (loss, logits_target)
    :
             (loss, outputs)  return_outputs  loss
```

prompt、response a、response bの最大長はそれぞれ2048に制限しています。したがって、入力の最大長は約6144です。

```
():
    preference_prompt_predict_template =

    preference_prompt_train_template = preference_prompt_predict_template +

    response_template =

    sep =
```

Post-Pretrain

post-pretrainには次のデータを使いました。post-pretrainは学習と同じ学習パイプラインを共有しています。

- ultrafeedback
- LMSYSの2位が収集したdataset

学習

学習には次のデータを使いました。

- LMSYSのデータ
- LMSYS 33k
- WSDM Cupのデータ

teacher model（Qwen2.5-72BとAthene-V2-Chat）は、年末年始の休暇中にdeepspeedを使って複数GPUで学習しました。learning rateの調整を避けるため、gradient_accumulation_stepsを1にして、batch sizeを単一GPU学習と同じに保ちました。

CV（distillationなし）

基本的に、14B modelにはlearning rate 10e-5が良いです。14e-5あたりからloss spikeが起き始めます。72B modelでは、より小さいlearning rateの方が良いです。

No	実験	Fold	lr	Accuracy
1	Qwen2.5 14B（post-pretrain、WSDMデータのみ）	0	7e-5	0.7055360462714315
2	-	0	8e-5	0.7007849617847552
3	-	0	9e-5	0.699958686221855
4	-	0	**10e-5**	**0.7051229084899814**
5	-	0	12e-5	0.7059491840528817
6	-	0	14e-5	0.5084693245197274
7	Phi4（WSDMデータのみ）	0	7e-5	0.6983061350960545
8	-	0	8e-5	0.6965502995248916
9	-	0	9e-5	0.6995455484404048
10	-	0	10e-5	0.6966535839702541
11	-	0	12e-5	0.6973765750877918
12	-	0	14e-5	0.7016112373476554

CV（distillationあり）

私の場合、最良のTemperatureは5.0（実験2〜5）、最良のsoft lossの重みは0.9（実験2〜7）でした。 LMSYSの1位にならってKLDivLossとCosineEmbeddingLossの平均も試しましたが、効果ははっきりしませんでした。KLDivLossで十分です。

No	実験	Fold	Accuracy	T	distil_weight	lr
1	Qwen2.5 14B（distillationなしのbaseline、WSDMデータのみ）	0	0.7051229084899814	-	-	10e-5
2	Qwen2.5 14B（distillation実験、WSDMデータのみ）	0	0.7061557529436067	1	0.5	10e-5
3	-	0	0.7082214418508572	10	0.5	10e-5
4	-	0	0.7076017351786821	3	0.5	10e-5
5	-	0	0.7102871307581078	5	0.5	10e-5

No	実験	Fold	Accuracy	T	distil_weight	lr
6	-	0	0.7121462507746333	5	0.7	10e-5
7	-	0	**0.7134889485643462**	**5**	**0.9**	10e-5
8	Qwen2.5 14B（teacher modelを増やしたdistillation、WSDMデータのみ）- 重みパラメータset1	0	0.7168973352613096	5	0.9	10e-5
9	Qwen2.5 14B（teacher modelを増やしたdistillation、WSDMデータのみ）- 重みパラメータset2	0	0.7164841974798596	5	0.9	10e-5

量子化

量子化にはauto-roundを使いました。重めの量子化設定にすると、精度の低下を抑えられます。量子化の実験はQwen2.5 3B modelで行いました。 最終的な量子化設定はseqlen=6144、nsamples=512、iters=3000です。

No	実験	seqlen	nsamples	iters	Fold	Accuracy
1	baseline - 量子化なし				0	0.681057
2	1 + lite設定	2048	256	500	0	0.674653
3	1 + 公式の最良設定	2048	512	1000	0	0.677752
4	3 + iter増加	2048	512	2000	0	0.678062
5	3 + iter増加	2048	512	3000	0	0.677649
6	4 + iter増加	2048	512	4000	0	0.673311 / 0.678785
7	3 + seqlen増加	4096	512	1000	0	0.676822
8	7 + iter増加	4096	512	2000	0	0.678888
9	8 + iter増加	4096	512	3000	0	0.673001
10	9 + iter増加	4096	512	4000	0	0.678888
11	10 + iter増加	4096	512	5000	0	0.680024
12	11 + seqlen増加	4096	512	6000	0	0.673104
13	7 + seqlen増加	6144	512	2000	0	0.678578
14	**13 + iter増加**	**6144**	**512**	**3000**	0	**0.680851 / 0.676513 / 0.681160 / 0.678578**
15	14 + iter増加	6144	512	4000	0	0.678682 / 0.674757

推論パート

推論にはvllmを使いました。推論パートに特別なことはなく、すべてengineeringです。次の工夫を使いました。

- テストデータのサイズに応じて、推論の制限時間を動的に変えます。学習フェーズでは推論の制限時間を265分、予測フェーズでは690分に設定しました。
- T4x2環境でmodelのthroughputをテストしました。modelの平均throughputは1分あたり約70000 tokenです。TTAでは、vllmのprefix cacheのおかげで、throughputは1分あたり約74000 tokenに達します。
- テストデータをtoken長でソートし、最初の25%のデータに対してQwen2.5 14B modelでTTAを行います。
- 残りの推論時間を監視し、throughputからPhi4で処理できるtoken数を割り出します。学習フェーズでは、テストデータの約60%を処理できます。

提出戦略

基本的には、leaderboard scoreが最も高い提出を選びます。leaderboard scoreが安定しないため、同じmodelを異なるseedで学習し、modelの重みを線形にmergeしました。 Phi4はLBで、短いpromptでは良く、長いpromptでは悪いことが分かりました。そこで、token長に応じてPhi4のensemble重みを変えました。

1. テストデータのprompt長の分位点を計算します。q1=0.25、q2=0.5、q3=0.75、q4=1.0
2. token長 <= q2 では、Phi4の重みを0.5にします。
3. token長 > q2 かつ <= q3 では、Phi4の重みを0.15にします。
4. token長 > q3 では、Phi4の重みを0.1にします。（推論の制限時間のため使っていません。学習フェーズでは、Phi4はテストデータの約60%しか推論できません。）

試したが諦めたこと

- Eediの1位に触発され、Qwen2.5-3bをdistillationして、prompt（=ユーザーからの依頼）の解き方についてCoTを生成させました。CVとLBは0.001〜0.002良くなりましたが、推論時間が長すぎて、timeout errorが頻繁に起きました。
- deepseekでdistillationされたQwen2.5-14bとQwen2.5-14b-1mも試しました。LBは良くありませんでした。

```
system_prompt_template =

user_prompt_template =
```

補足Q&A

cloudwantyou@gmail.com — コメント

ご説明ありがとうございます。私たちもCoTを試しましたが、GPT-4oに多くの資源を費やしても改善はかなり限られていました。CoTデータには、DeepSeek-R1と同様のreinforcement learningのアプローチを採用する必要があるのかもしれません。いずれにせよ、おめでとうございます！

RihanPiggy（著者） — 回答

ありがとうございます！あなたもおめでとうございます！ まさにその通りで、SFTによるdistillationでは十分ではありません。私はllama3-405Bを使ってteacherのCoTを生成し、Qwen2.5-3Bを学習しました。改善がわずかだったことに加えて、もう1つの問題は、SFTの後、Qwen2.5-3Bの生成長がdistillation前と比べて爆発的に長くなり、推論時間が大幅に増えたことです…

Miles — 質問

おめでとうございます！TTAをどのように行ったか、詳しく教えていただけますか。やり方はいろいろあるので。

RihanPiggy（著者） — 回答

ありがとうございます。単純にresponse AとBを入れ替えました。prompt全体のtoken長でdataframeをソートし、データの最初の25%にTTAを行いました。 推論コードを公開しました。run_inference.pyを見てみてください。